

## Assignment 2 Code Analysis:

- A. Used Google Gemini and Anthropic Claude
- B. I accessed Gemini through Google search (with SearchAI) and I went to Claude's website to generate the c file.
- C. Prompt: I want you to create a C file program, this program will prioritize emails for a busy company CEO. You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status. You must implement a MaxHeap using a list-based implementation. Then use MaxHeap to handle all your email prioritizing for the CEO. You must create functions from scratch. Do not include pre-existing heap modules. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. Assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Commands are Email <sender category>, <subject line>, <date> which are meant to rank whoever is sending the email in the heap as follows:

Boss - Read first

Subordinate - Read next

Peer - Read next

ImportantPerson - Read next

OtherPerson - Read last

If there's more than one email from the sender, the newest email (not oldest) should be read first. EMAIL is followed by space, other fields are delimited, also assume sender category is one of the 5 senders listed above, assume <subject line> is a string that contains spaces (no commas), and assume <date> is in the format: MM-DD-YYYY

NEXT gives the email for the CEO to read, which displays the information on terminal in following format:

"Sender:

Subject:

Date: "

READ has the CEO reading the email and removing it from the heap and COUNT displays the current count of unread emails. This is what input/output looks like:

- Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- Your program should display this:

```
There are 6 emails to read.

Next email:
    Sender: Boss
    Subject: Never Mind
    Date: 01-03-2025

Next email:
    Sender: Boss
    Subject: Important
    Date: 12-20-2024

There are 4 emails to read.
```

D. Gemini Code:

[https://github.com/Somatron/EECS-348-Assignment-2/blob/main/Gemini\\_CEO\\_Email.c](https://github.com/Somatron/EECS-348-Assignment-2/blob/main/Gemini_CEO_Email.c)

Claude's Code:

[https://github.com/Somatron/SWE-ASSIGNMENTS/blob/main/Claude\\_CEO\\_Email.c](https://github.com/Somatron/SWE-ASSIGNMENTS/blob/main/Claude_CEO_Email.c)

- E. Both of Gemini's and Claude's code includes  $O(n)$  space which makes sense because the more you add to the heap the bigger the space will be. As for time complexity, it kinda varies, for example adding and deleting both include  $O(\log n)$  because of reordering the heap, while peeking takes  $O(1)$  since you're only checking the size of the heap. The code correctness seems right, and as for maintainability, Claude's comments are organized but seem scattered all over the place (some areas have ton of commenting while other areas like the MaxHeap have little to no comment

at all) while Google's code is extremely well formatted, simplistic, and also explains/summarizes each block of code easily.

- F. Even though Claude's output code is extremely well organized and has good memory management safety, I still prefer Gemini's code due to its simplicity and being easier to understand.
- G. I improved the output of the code to be more better formatted, also I added a command line interface where you type a text file input into the program and generates the output (similar to python) instead of running both the program and the file simultaneously. There wasn't anything I could improve on time and space complexity because adding to the heap is already  $O(\log n)$ . Sometimes it's  $O(1)$  if it's a small small number we don't need to move it at all and the space complexity of the heap is  $O(n)$  at best.